



Komplexer Build-Prozess

Tim Schmitz
16.05.2026

Inhalt

Übersicht	3
Die Properties-Datei	3
Die Build-Datei	4

Ein komplexerer Build-Prozess

Hier wird ein komplexerer Build-Prozess vorgestellt, der auf dem Apache Build-Tool ant basiert. Dieser Build-Prozess wird, anders als der oben vorgestellte einfache Build-Prozess, aus der Entwicklungsumgebung heraus gestartet. Wie zuvor auch wird die kompilierte Anwendung in eine Java Archive-Datei verpackt und diese dann in die Produktivumgebung deployet.

Übersicht

Für den lokalen Build- und Deployment-Prozess benötigt man zwei Dateien:

1. Eine Datei build.properties, um die Eigenschaften separat zu definieren
2. Eine Datei build.xml, die das Build-File für den Prozess definiert

Das Zusammenspiel der Dateien mit dem Ant Build-Prozess sieht man in der folgenden Abbildung:

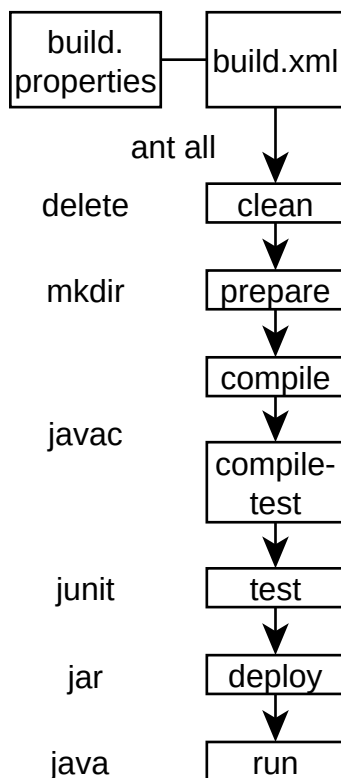


Abbildung 1: Komplexer Build-Prozess

Die Properties-Datei

Die Properties-Datei build.properties definiert die folgenden Parameter, die umgebungsabhängig sind. Im Beispiel wurde eine Umgebung des Users "test" verwendet.

```
#-----#
# Allgemeine Parameter #
#-----#
projects      = C:/Users/test/Daten/Java/Projects
runtime      = C:/Users/test/Daten/Java/Applications
libraries    = C:/Users/test/Daten/Java/Libraries

#-----#
# Projekt-Parameter   #
#-----#
project.application = ${projects}/application
project.source     = ${project.application}/src
project.properties = ${project.application}/properties
```

```

project.metainf      = ${project.application}/META-INF

#-----#
# Laufzeit-Parameter #
#-----#
runtime.application = ${runtime}/application
runtime.build       = ${runtime.application}/build
runtime.properties  = ${runtime.application}/properties
runtime.metainf     = ${runtime.application}/META-INF

#-----#
# Bibliotheken       #
#-----#
log4j.api           = ${libraries}/apache-log4j-2.11.1-bin
junit.api           = ${libraries}/junit-5

#-----#
# Test-Parameter    #
#-----#
project.test        = ${project.application}/test
runtime.test        = ${runtime.application}/test
runtime.output      = ${runtime.application}/output

```

Allgemeine Parameter

Die allgemeinen Parameter sind die Parameter, die als Basisverzeichnisse für die einzelnen Parameter definiert wurden. In diesem Fall ist dies die Entwicklungsumgebung (projects), die Ablauf- oder Laufzeitumgebung (runtime) und die Bibliotheksumgebung (libraries).

Projekt-Parameter

Projekt-Parameter sind zunächst einmal der Pfad zum Projekt selbst (xdoc), dann das Quellcode-Verzeichnis (source) sowie die Properties-Umgebung und die Meta-Informationen, die zur Entwicklungsumgebung gehören.

Laufzeit-Parameter

Laufzeit-Parameter sind der Ausführungspfad zur Anwendung (xdoc), die Umgebung für die Übersetzung der Anwendung (build) sowie die Properties-Umgebung und die Meta-Informationen mit der Manifest-Datei, die für das ausführbare Java-Archiv benötigt wird.

Bibliotheken

Die verwendeten Bibliothekspfade liegen alle in einem zentralen Verzeichnis, das neben der Entwicklungs- und der Ablaufumgebung liegt. Die verwendeten APIs wie Log4j2, Junit, Batik und FOP werden hierüber referenziert.

Test-Parameter

Test-Parameter werden benötigt, um die Test-Umgebung der Entwicklungsumgebung wie auch für den finalen Test der Ablaufumgebung zu referenzieren mitsamt des Orts, an dem die Testresultate geschrieben werden.

Die Build-Datei

Die Build-Datei build.xml definiert verschiedene Ziele, sogenannte Targets, und hat folgendes Aussehen:

```

<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<project name="application" default="all" basedir=".">

  <!-- ***** -->
  <!-- Verwendung des Property-Files hängt von der Umgebung ab -->
  <!-- ***** -->
  <property file="properties/build.properties"/>

  <!-- ***** -->
  <!-- General Target -->
  <!-- ***** -->
  <target name="all" depends="clean,prepare,compile,compile-test,test,deploy,run"/>

  <!-- ***** -->
  <!-- Target clean löscht die Artefakte der Laufzeitumgebung -->
  <!-- ***** -->
  <target name="clean">
    <delete dir="${runtime.build}"/>

```

```

<delete dir="${runtime.properties}"/>
<delete dir="${runtime.metainf}"/>
<delete dir="${runtime.test}"/>
<delete dir="${runtime.output}"/>
<delete file="${runtime.application}/application.jar"/>
</target>

<!-- ***** -->
<!-- Target prepare bereitet die Artefakte der Laufzeitumgebung vor -->
<!-- ***** -->
<target name="prepare" depends="clean">
  <mkdir dir="${runtime.build}"/>
  <mkdir dir="${runtime.properties}"/>
  <mkdir dir="${runtime.metainf}"/>
  <mkdir dir="${runtime.test}"/>
  <mkdir dir="${runtime.output}"/>
</target>

<!-- ***** -->
<!-- Target compile übersetzt die Quellen in Klassenobjekte -->
<!-- ***** -->
<target name="compile" description="Übersetzung der Quellen" depends="prepare">
  <javac srcdir="${project.source}" destdir="${runtime.build}" includeantruntime="false" deprecation="off">
    <classpath>
      <pathelement path="${runtime.build}"/>
      <pathelement location="${log4j.api}/log4j-api-2.11.1.jar"/>
      <pathelement location="${log4j.api}/log4j-core-2.11.1.jar"/>
    </classpath>
  </javac>
</target>

<!-- ***** -->
<!-- Target compile-test übersetzt die Testtreiber -->
<!-- ***** -->
<target name="compile-test" depends="compile">
  <javac srcdir="${project.test}" destdir="${runtime.test}" includeantruntime="false">
    <classpath>
      <pathelement location="${junit.api}/junit-jupiter-api-5.6.1.jar"/>
      <pathelement path="${runtime.build}"/>
    </classpath>
  </javac>
</target>

<!-- ***** -->
<!-- Target test führt die Tests durch -->
<!-- ACHTUNG: die jupiter- und junit-apis werden in den ant/lib-Ordner von IntelliJ kopiert -->
<!-- ***** -->
<target name="test" depends="compile-test">
  <junitlauncher>
    <classpath>
      <pathelement path="${junit.api}"/>
      <pathelement path="${runtime.test}"/>
      <pathelement path="${runtime.build}"/>
    </classpath>
    <testclasses outputdir="${runtime.output}">
      <fileset dir="${runtime.test}"/>
      <listener type="legacy-brief" sendSysOut="true"/>
      <listener type="legacy-xml" sendSysErr="true" sendSysOut="true"/>
    </testclasses>
  </junitlauncher>
</target>

<!-- ***** -->
<!-- Target deploy sorgt für die Auslieferung der Anwendung in die Laufzeitumgebung -->
<!-- ***** -->
<target name="deploy" description="Auslieferung der Anwendung" depends="compile">
  <copy file="${project.properties}/application.properties" todir="${runtime.properties}"/>
  <copy file="${project.properties}/log4j2.xml" todir="${runtime.properties}"/>
  <copy file="${project.metainf}/MANIFEST.MF" todir="${runtime.metainf}"/>
  <jar destfile="${runtime.application}/application.jar">
    <basedir="${runtime.build}">
    <manifest="${runtime.metainf}/MANIFEST.MF">
  </jar>
</target>

<!-- ***** -->
<!-- Target run startet die Anwendung in der Laufzeitumgebung -->
<!-- ***** -->
<target name="run" description="Ausführung der Anwendung" depends="deploy">
  <java jar="${runtime.application}/application.jar" fork="yes">
    <sysproperty key="log4j.configurationFile" value="file://${runtime.properties}/log4j2.xml"/>
  </java>
</target>

</project>

```

Das Ant Build-File gehorcht der Nomenklatur für XML-Dateien. Wichtig ist die Kodierung im universellen Zeichensatz UTF-8, die auch in den Targets referiert wird. Die verschiedenen Targets, mit denen das Build-Tool Apache Ant gerufen werden kann, haben dabei folgende Bedeutung:

Target all

Dieses Target umspannt alle anderen Targets, das heißt mit ant all wird die komplette Kommandokette von der Säuberung der Verzeichnisse bis hin zum Start der Anwendung durchlaufen.

Target clean

Dieses Target dient zur Säuberung der Ablaufumgebung. Die Zielverzeichnisse werden gelöscht. Dazu wird die Ant Task delete verwendet.

Target prepare

Dieses Target dient zur Vorbereitung der Ablaufumgebung. Die Zielverzeichnisse werden erzeugt. Dazu wird das Ant Task mkdir verwendet.

Target compile

Das Target übersetzt die Quellen in Klassenobjekte und verwendet die Zeichensatz-Kodierung UTF-8. Dazu wird die Ant Task javac verwendet. Die Klassenobjekte werden in das Build-Zielverzeichnis der Laufzeitumgebung geschrieben. Im Klassenpfad wird das Build-Zielverzeichnis als Pfadelement mitaufgeführt wie auch alle Bibliotheken, die für die Übersetzung notwendig sind.

Target compile-test

Dieses Target übersetzt die Quellen der Testklassen und schreibt sie in das Zielverzeichnis. Im Gegensatz zur Quellcode-Übersetzung wird hier nur eine andere externe Bibliothek als Pfadelement verwendet, nämlich die Testbibliothek JUnit für Java.

Target test

Dieses Target führt die Tests auf den übersetzten Testklassen mittels des Ant Tasks junitlauncher durch. Dabei werden im Klassenpfad die Klassenverzeichnisse der Quellcode-Klassen und der Testklassen referenziert. Die Testergebnisse werden in das Ausgabeverzeichnis geschrieben. Als Vorbereitung müssen die jupiter- und junit-apis werden in den ant/lib-Ordner von IntelliJ kopiert werden.

Target deploy

Dieses Target sorgt für die Auslieferung der Anwendung in die Laufzeitumgebung. Dazu wird mehrfach die Ant Task copy verwendet, um die Ausführung der Anwendung vorzubereiten, indem die Anwendungs-Properties, die Logger-Konfiguration und das Manifest in die richtigen Verzeichnisse der Zielumgebung geschrieben werden. Anschließend wird die Ant Task jar benutzt, um das Java-Archiv mit der korrekten UTF-8-Kodierung und dem inkludierten Manifest zu erstellen.

Target run

Dieses Target startet die Anwendung in der Laufzeitumgebung. Es benutzt die Ant Task java und wird mit dem Namen des Verzeichnisses als Argument aufgerufen, auf dem die Anwendung operiert. Hierbei muss wieder auf die korrekte Zeichensatz-Kodierung der Java Virtual Machine geachtet werden sowie die Übergabe der korrekten Konfigurationsdatei für den Apache Logger.